Table des matières

Cours 05 — Tester, historiser et documenter 1
Ce que vous allez faire 1
Avant de commencer 2
Étape 1 — Les tests génériques · MARTS 2
Étape 2 — Le grain et le témoin, en tests singuliers · MARTS 5
Étape 3 — L'histoire des prix, `snp_prix` · STAGING 6
Étape 4 — Le prix d'époque · INTERMEDIATE 9
Étape 5 — Documenter · Fiabiliser et faire tourner 10
Étape 6 — Tout d'un geste, `dbt build` · Fiabiliser et faire tourner 11
Étape 7 — Versionner · Fiabiliser et faire tourner 11
À retenir 12

Cours 05 — Tester, historiser et documenter

Cours 5/9 · La Brûlerie — Formation Analytics Engineering

STAGING	**INTERMEDIATE**	**MARTS**
reprendre la `RAW`, une vue par table	joindre, clarifier, préparer l'étoile	concevoir et livrer les étoiles
✓ ici · Fiabiliser et faire tourner, ce qui traverse les trois couches : les tests, l'incrémental, l'intégration continue	✓	✓

Ce que vous allez faire

L'étoile du cours 04 tient, et deux contrôles faits à la main l'ont prouvé, une fois. Ce cours les rend permanents : des tests que dbt relance à chaque commande, déclarés en YAML ou écrits en SQL, dont le témoin de la caisse, ticket par ticket. Puis il rend au catalogue son histoire : un snapshot garde chaque prix que la caisse a connu depuis 2022, et un dernier test vérifie que chaque ligne vendue l'a été au prix en vigueur ce jour-là. Le cours finit par la documentation, tirée des descriptions que vous aurez écrites, et par `dbt build`, qui bâtit, historise et teste d'un geste.

© 2026 iAlexMG. Tous droits réservés.

l'étoile du cours 04 → ses tests, génériques puis singuliers → `snp_prix`, l'histoire des prix → le prix d'époque → la documentation et `dbt build`

Les fichiers à écrire se donnent en pseudocode, et le code complet est dans `Codes_05_AE.md`, à ouvrir après avoir essayé.

Avant de commencer

Deux fenêtres de l'Anaconda Prompt servent à ce cours, toutes deux placées dans `Projet_Brulerie` : la première pour le Bloc-notes, dbt et Git ; la seconde pour le notebook de DuckDB, qui s'ouvre à l'étape 3. Dans la première :

```
conda activate formations
REM Attendu : (formations) au début de l'invite.
set DBT_PROJECT_DIR=02_ae
REM Attendu : rien ne s'affiche.
dbt run
REM Attendu : 9 of 9, cinq vues et quatre tables, puis Done. PASS=9 WARN=0 ERROR=0.
```

Ce cours part du projet tel que le cours 04 l'a laissé, et `dbt run` rebâtit l'étoile : les tests liront ses tables. S'il manque un modèle, reprenez la fin du cours 04 avant de continuer.

Étape 1 — Les tests génériques · MARTS

Un test cherche une faute. C'est une requête qui rend les lignes fautives, et dbt dit le test réussi quand elle n'en rend aucune. Les contrôles du cours 04 se lisaient à l'œil, une fois ; un test se relance à chaque commande, et il échoue tout seul le jour où un modèle se trompe. Les plus courants sont génériques : on les déclare dans un fichier YAML, à côté des modèles, et dbt écrit la requête.

Le test	Ce qu'il vérifie	Où ici
`unique`	aucune valeur deux fois dans la colonne	la clé de chaque dimension
`not_null`	aucune valeur vide	les clés, la catégorie du produit
`accepted_values`	chaque valeur fait partie d'une liste fermée	la catégorie du produit, six valeurs
`relationships`	chaque valeur existe dans une colonne d'un autre modèle	chaque clé du fait, dans sa dimension

Deux de ces tests ignorent les valeurs vides. `accepted_values` d'abord : un décodage entièrement raté, qui laisserait la colonne vide, passerait au vert. Il va donc toujours avec un `not_null` à côté. `relationships` aussi, et c'est voulu pour `cle_membre` : un ticket sans carte a une clé vide, le cours 02 l'a décidé, et le test ne vérifie que les clés remplies.

© 2026 iAlexMG. Tous droits réservés.

Le fait recalcule ses clés par `md5`, sans les chercher dans ses dimensions. C'est simple, mais rien ne garantit alors qu'une clé du fait existe dans sa dimension : un produit vendu et absent du catalogue donnerait une clé orpheline, et la jointure par catégorie du cours 04 perdrait sa vente sans un mot. `relationships` ferme cette porte.

Le même fichier porte les descriptions des modèles et des colonnes : c'est la documentation, que l'étape 5 publiera. Deux points de forme. Le bloc des tests s'appelle `data_tests:`. Et les paramètres d'un test (la liste de valeurs, le modèle visé) vont sous `arguments:` : sans lui, dbt fonctionne, mais il affiche à chaque commande un avertissement de dépréciation, `MissingArgumentsPropertyInGenericTestDeprecation`. Beaucoup d'exemples en ligne ont encore l'ancienne forme.

1.1 Écrire le fichier, dans la première fenêtre.

```
notepad 02_ae\models\marts\_marts.yml
```

Accepter la création, puis écrire à partir de ce pseudocode. Les six catégories sont dans le dictionnaire des sources, section 2, « Les codes du catalogue » : chacune entre apostrophes, séparées par des virgules.

```
# 02_ae/models/marts/_marts.yml : l'étoile des ventes, décrite et testée
models:
  # 1. les dimensions : une clé unique et toujours remplie
  - name: dim_date
    description: "Les 30 jours de juin 2025, un par ligne, fabriqués sans source."
    columns:
      - name: date_jour
        description: "La clé : la date elle-même."
        data_tests:
          - ____
          - ____
  - name: dim_produit
    description: "Un produit du catalogue par ligne, au 2025-06-30."
    columns:
      - name: cle_produit
        description: "La clé de substitution, md5 du numéro du produit."
        data_tests:
          - unique
          - not_null
      # 2. une liste fermée, et son not_null à côté
      - name: categorie
        data_tests:
          - not_null
          - accepted_values:
              arguments:
                values: [____]
```


© 2026 iAlexMG. Tous droits réservés.

```yaml
    # puis, à écrire : dim_membre, sa description, et sa clé cle_membre avec les deux mêmes tests
    # 3. le fait : chaque clé recalculée existe dans sa dimension
    - name: fct_ventes
      description: "Le fait des ventes, une ligne par ligne d'un ticket payé."
      columns:
        - name: cle_date
          data_tests:
            - not_null
            - relationships:
                arguments:
                  to: ref('____')
                  field: ____
        # puis, à écrire : cle_produit, avec les deux mêmes tests, vers la clé de dim_produit
        - name: cle_membre
          description: "Vide pour un ticket sans carte de membre."
          data_tests:
            # seulement relationships, vers la clé de dim_membre : la clé peut rester vide
```

Enregistrer, puis fermer le Bloc-notes.

1.2 Lancer les tests. `dbt test` lance chaque test du projet.

```
dbt test
REM Attendu : 13 tests, chacun PASS, puis Done. PASS=13 WARN=0 ERROR=0.
```

dbt nomme chaque test d'après son modèle et sa colonne : `unique_dim_produit_cle_produit`, `relationships_fct_ventes_cle_date__date_jour__ref_dim_date_`. Le nom n'est pas beau, mais le jour où un test échoue, il dit seul quelle colonne de quel modèle a fauté.

Si dbt s'arrête sur	**La cause, et le geste**
`Parsing Error`, `Error reading brulerie: marts\_marts.yml`, `Syntax error near line …`	une indentation décalée : chaque niveau compte deux espaces de plus que celui qui le contient, comme dans le pseudocode ; la ligne citée est souvent celle qui suit la faute
`[WARNING][MissingArgumentsPropertyInGenericTestDeprecation]`	les paramètres d'un test ne sont pas sous `arguments:` : les y mettre, deux espaces plus à droite
`IO Error: Cannot open file "…\brulerie.duckdb"`	le notebook tient l'entrepôt : quitter DuckDB dans la seconde fenêtre, puis relancer dbt

© 2026 iAlexMG. Tous droits réservés.

Étape 2 — Le grain et le témoin, en tests singuliers · MARTS

Un test singulier est une requête que vous écrivez vous-même, dans le dossier `tests` du projet. Comme un test générique, elle rend les lignes fautives, et dbt réussit le test quand elle n'en rend aucune. Les deux contrôles du cours 04 y passent. Le grain d'abord : la requête de l'étape 6.5, telle quelle, le fait lu par `ref()`.

Le témoin ensuite. Le cours 04 comparait deux totaux, 16 718,00 $ des deux côtés. Un total peut tomber juste sur deux erreurs qui se compensent : un ticket de trop d'un côté, un ticket du même montant perdu de l'autre. Le témoin passe donc au ticket : pour chaque ticket payé, la somme de ses lignes dans le fait doit égaler le sous-total que la caisse a imprimé.

Pour remplir	**L'outil**
nommer un résultat intermédiaire	`with fait as (…), caisse as (…)` : une expression de table commune, qu'on lit ensuite comme une table
comparer les deux côtés	`full outer join` : chaque ticket des deux côtés, même présent d'un seul
prendre le numéro du côté qui l'a	`coalesce(f.ticket_id, c.ticket_id)`

La jointure externe complète est le cœur du test. Une jointure interne ne garderait que les tickets présents des deux côtés : un ticket perdu par le fait ne produirait aucune ligne, et le test passerait. L'expression de table commune s'apprend au Cours 05 de la formation SQL avancé, `Theorie_05_SQL.md`, section 2.

2.1 Créer le dossier des tests, puis écrire le test du grain.

```
mkdir 02_ae\tests
notepad 02_ae\tests\assert_grain_fct_ventes.sql
```

Accepter la création, puis écrire. Pas de point-virgule à la fin : dbt place la requête dans la sienne, qui compte les lignes rendues.

```
-- 02_ae/tests/assert_grain_fct_ventes.sql : aucun couple ticket et ligne deux fois dans le fait
-- 1. les couples (ticket, ligne) que le fait porte plus d'une fois : chaque ligne rendue est un échec
select ticket_id, ligne, count(*) as fois
from {{ ref('____') }}
group by ticket_id, ligne
having count(*) > 1
```

Enregistrer, puis fermer le Bloc-notes.

2.2 Écrire le témoin.

```
notepad 02_ae\tests\assert_temoin_caisse.sql
```

Accepter la création, puis écrire :

© 2026 iAlexMG. Tous droits réservés.

```sql
-- 02_ae/tests/assert_temoin_caisse.sql : chaque ticket payé, au cent près, dans le fait et à la caisse
with fait as (
    -- 1. le montant de chaque ticket, additionné depuis les lignes du fait
    select ticket_id, sum(montant) as montant
    from {{ ref('fct_ventes') }}
    group by ticket_id
),
caisse as (
    -- 2. le sous-total de chaque ticket payé, tel que la caisse l'a imprimé
    select ticket_id, sous_total
    -- puis, à écrire : lire stg_tickets par ref(), et ne garder que les tickets payés
)
-- 3. un ticket d'un seul côté, ou deux montants qui diffèrent : chaque ligne rendue est un échec
select coalesce(f.ticket_id, c.ticket_id) as ticket_id, f.montant, c.sous_total
from fait as f
____ join caisse as c on c.ticket_id = f.ticket_id
where f.ticket_id is null or ____ or f.montant <> c.sous_total
```

Enregistrer, puis fermer le Bloc-notes. L'égalité est stricte, sans tolérance : les montants sont des décimaux depuis le cours 03, et ils comptent au cent près.

2.3 Lancer les tests.

```
dbt test
REM Attendu : 15 tests, dont assert_grain_fct_ventes et assert_temoin_caisse, puis Done. PASS=15.
```

Si dbt s'arrête sur	La cause, et le geste
`Parser Error: syntax error at or near ";"`	un point-virgule à la fin du test : l'enlever
`FAIL 1 assert_temoin_caisse`, ou un autre nombre	autant de tickets en faute : le message donne la requête compilée, dans `02_ae\target\compiled\…`, à relancer dans le notebook pour les voir

Étape 3 — L'histoire des prix, `snp_prix` · STAGING

`dim_produit` montre le catalogue d'aujourd'hui : quand un prix change, le nouveau écrase l'ancien. On appelle type 1 cette façon de faire, qui ne garde que le présent. Or la caisse a changé ses prix deux fois : le 1er février 2024, les boissons chaudes et les viennoiseries ont pris 0,25 $; le 3 mars 2025, les boissons ont repris 0,25 $, et les sacs de grains, 2,00 $. Le catalogue du 30 juin ne garde que le dernier prix. Le journal des modifications garde tout : 118 lignes depuis l'installation de la caisse, en novembre 2022, dont 32 créations de produits et 38 changements de prix.

© 2026 iAlexMG. Tous droits réservés.

Un snapshot est l'outil de dbt pour garder l'histoire. À chaque passage, il compare le résultat de sa requête à ce qu'il a déjà gardé : quand une colonne suivie change, il ferme l'ancienne version en remplissant sa colonne `dbt_valid_to`, et en ouvre une nouvelle, datée dans `dbt_valid_from`. La version en vigueur est celle dont `dbt_valid_to` est vide. Une ligne par version : c'est le type 2.

En entreprise, un snapshot passe chaque nuit sur l'export du catalogue, et l'histoire commence la première nuit. Ici, le journal permet de refaire le catalogue tel qu'il était n'importe quel jour : la requête prend, de chaque produit, le dernier prix connu au jour regardé. Le snapshot regardera quatre fois : à la fin de 2022, de 2023 et de 2024, puis le 30 juin 2025.

Le réglage	**Ce qu'il dit**
`unique_key='produit_id'`	ce qui désigne la même chose d'une version à l'autre : le produit
`strategy='check'`, `check_cols=['prix']`	ouvrir une version quand le prix change, et seulement alors
`updated_at='en_vigueur_le'`	dater la version du jour où le prix est entré en vigueur ; sans lui, dbt la daterait de l'instant où la commande tourne
`var("jour", "2025-06-30")`	le jour regardé, donné à la commande par `--vars "{jour: 2024-12-31}"` ; le second argument est sa valeur de départ, quand la commande n'en donne pas

Sans valeur de départ, `dbt build` et la documentation, qui compilent tout le projet, s'arrêteraient sur `Required var 'jour' not found in config`. dbt veut aussi un horodatage pour dater ses versions, pas une date : avec une date, il avertit à chaque passage. Le café change ses prix à 6 h 41, avant d'ouvrir à 7 h : un prix vaut pour toute la journée, et `date_trunc('day', …)` ramène l'heure à minuit. Enfin, dans un snapshot, les commentaires s'écrivent `{# … #}`, ceux de Jinja : dbt les retire avant d'envoyer la requête.

3.1 Déclarer le journal comme source, dans la première fenêtre. Une source se déclare le jour où quelque chose la lit.

```
notepad 02_ae\models\staging\_sources.yml
```

Ajouter cette entrée au bout de la liste des tables, alignée sur les quatre autres :

```
      - name: raw_modifications
        description: "Le journal des modifications du catalogue, une ligne par valeur changée, depuis 2022."
```

Enregistrer, puis fermer le Bloc-notes.

3.2 Créer le dossier des snapshots, puis écrire le snapshot.

```
mkdir 02_ae\snapshots
notepad 02_ae\snapshots\snp_prix.sql
```

Les codes du champ modifié sont dans le dictionnaire des sources, section 2, « `champ` des modifications ». Accepter la création, puis écrire :

© 2026 iAlexMG. Tous droits réservés.

```
{# 02_ae/snapshots/snp_prix.sql : le prix de chaque produit, une version par prix, tirée du journal #}
{% snapshot snp_prix %}
{{ config(
    unique_key='produit_id', strategy='check', check_cols=['____'], updated_at='en_vigueur_le'
) }}
select
    {# 1. le produit, son prix, et le jour où ce prix est entré en vigueur #}
    produit_id::integer as produit_id,
    nouvelle_valeur::decimal(10, 2) as prix,
    date_trunc('____', horodatage::timestamp) as en_vigueur_le
from {{ source('brulerie', '____') }}
{# 2. le catalogue au jour regardé : les créations et les changements de prix connus ce jour-là #}
where champ in ('____', '____')
    and horodatage::date <= '{{ var("jour", "2025-06-30") }}'
{# 3. de chaque produit, le dernier prix connu #}
qualify row_number() over (partition by ____ order by horodatage ____) = 1
{% endsnapshot %}
```

Enregistrer, puis fermer le Bloc-notes.

3.3 Prendre les quatre regards, dans l'ordre. `dbt snapshot` ne lance que les snapshots.

```
dbt snapshot --vars "{jour: 2022-12-31}"
REM Attendu : 1 of 1 OK snapshotted main.snp_prix, puis Done. PASS=1.
dbt snapshot --vars "{jour: 2023-12-31}"
dbt snapshot --vars "{jour: 2024-12-31}"
dbt snapshot --vars "{jour: 2025-06-30}"
REM Attendu : chaque fois, 1 of 1 OK snapshotted main.snp_prix.
```

L'ordre compte. Un snapshot compare au présent ce qu'il a gardé, et ne revient jamais en arrière : un regard ancien pris après un récent ouvrirait des versions datées d'avant celles qu'il ferme.

Si dbt s'arrête sur	**La cause, et le geste**
`depends on a source named 'brulerie.raw_modifications' which was not found`	le journal n'est pas déclaré : reprendre l'étape 3.1
Les regards pris dans le désordre : des versions qui se ferment avant de s'ouvrir, et `assert_prix_d_epoque` en échec à l'étape 4	le snapshot ne se défait pas : quitter le notebook, effacer la table par `duckdb plateforme\entrepot\brulerie.duckdb -c "drop table snp_prix"`, puis reprendre les quatre regards dans l'ordre

3.4 Ouvrir le notebook. Seconde Anaconda Prompt : activer `formations`, se placer dans `Projet_Brulerie`, puis lancer DuckDB.

```
duckdb -ui
```


© 2026 iAlexMG. Tous droits réservés.

Créer un notebook et le renommer « AE Cours 05 », puis attacher l'entrepôt dans la première cellule :

```
attach if not exists 'plateforme/entrepot/brulerie.duckdb' as brulerie (read_only);
-- Attendu : brulerie paraît sous Attached databases, avec snp_prix.
```

3.5 Compter les versions, dans une cellule neuve. `count(dbt_valid_to)` ne compte que les valeurs remplies : les versions fermées.

```
select count(*) as versions, count(dbt_valid_to) as fermees
from brulerie.snp_prix;
-- Attendu : 70 versions, dont 38 fermées ; les 32 autres, une par produit, sont en vigueur.
```

Les 38 versions fermées répondent aux 38 changements de prix du journal : chaque hausse a fermé une version et en a ouvert une autre.

3.6 Suivre le Latte, le produit le plus vendu du mois, le numéro 105 du catalogue.

```
select prix, dbt_valid_from, dbt_valid_to
from brulerie.snp_prix
where produit_id = 105
order by dbt_valid_from;
-- Attendu : 3 versions : 5.25 du 2022-11-14 au 2024-02-01, 5.50 jusqu'au 2025-03-03, puis 5.75,
-- en vigueur (dbt_valid_to vide).
```

Une version se ferme le jour même où la suivante s'ouvre : le 2024-02-01 est la fin de la première et le début de la deuxième.

3.7 Lire le prix du Latte au 15 juin 2024. La version en vigueur un jour donné s'est ouverte au plus tard ce jour-là, et ne s'est pas encore fermée. La fin est exclue : autrement, une vente du 1er février 2024, le jour où l'une se ferme et l'autre s'ouvre, trouverait deux versions.

```
-- 1. le prix du Latte en vigueur le 15 juin 2024
select prix
from brulerie.snp_prix
where produit_id = 105
  and dbt_valid_from <= date '2024-06-15'
  and (dbt_valid_to is null or date '2024-06-15' ____ dbt_valid_to);
-- Attendu : 5.50.
```

3.8 Fermer le notebook, pour que dbt puisse écrire dans l'entrepôt : fermer l'onglet du navigateur, puis quitter DuckDB dans la seconde Anaconda Prompt.

```
.quit
```

Attendu : l'invite revient, et la seconde fenêtre peut se fermer.

Étape 4 — Le prix d'époque · INTERMEDIATE

© 2026 iAlexMG. Tous droits réservés.

Le prix d'époque est celui qui était en vigueur le jour de la vente. Chaque ligne d'un ticket porte le prix que la caisse a imprimé ; le snapshot dit le prix du catalogue ce jour-là. Les deux viennent de systèmes différents, et ils doivent tomber d'accord : c'est le troisième test singulier. Il lit `int_ventes`, qui porte le produit, la date et le prix de chaque ligne vendue, et joint chaque ligne à la version de son produit en vigueur ce jour-là, par la condition de l'étape 3.7. Un snapshot se lit par `ref()`, comme un modèle.

La jointure est externe à gauche : une ligne qui ne trouve aucune version (un produit vendu avant sa création, ou absent du journal) garde un prix vide, et le test la rend comme une faute, au même titre qu'un prix qui diffère.

4.1 Écrire le test, dans la première fenêtre.

```
notepad 02_ae\tests\assert_prix_d_epoque.sql
```

Accepter la création, puis écrire :

```
-- 02_ae/tests/assert_prix_d_epoque.sql : chaque ligne vendue au prix en vigueur le jour de la vente
select v.ticket_id, v.ligne, v.date_vente, v.prix_unitaire, p.prix
-- 1. chaque ligne vendue, avec la version du prix de son produit en vigueur ce jour-là
from {{ ref('int_ventes') }} as v
left join {{ ref('____') }} as p
    on p.produit_id = v.produit_id
    -- 2. ouverte au plus tard le jour de la vente, et pas encore fermée ce jour-là
    and p.dbt_valid_from <= v.date_vente
    and (____)
-- 3. une ligne sans version, ou vendue à un autre prix : chaque ligne rendue est un échec
where p.prix is null or ____
```

Enregistrer, puis fermer le Bloc-notes.

4.2 Lancer les tests.

```
dbt test
REM Attendu : 16 tests, dont assert_prix_d_epoque, puis Done. PASS=16 WARN=0 ERROR=0.
```

Les 2 509 lignes du mois ont été vendues au prix d'époque, les 24 boissons offertes comprises : le rabais s'ajoute à la ligne sans toucher au prix. En juin, ce prix est aussi celui d'aujourd'hui, puisque la dernière hausse date du 3 mars. Le test tiendra le jour où un mois chevauchera une hausse, là où une comparaison au seul catalogue du 30 juin se tromperait.

Étape 5 — Documenter · Fiabiliser et faire tourner

La documentation d'un projet dbt s'écrit dans les mêmes fichiers YAML que ses tests et ses sources : les descriptions de l'étape 1, celles des sources du cours 03. dbt en fait un site, avec le graphe des modèles qu'il tire de vos `ref()` et de vos `source()` : personne ne l'a dessiné.

5.1 Générer la documentation. `--static` réunit tout dans une seule page, qui s'ouvre sans serveur.

© 2026 iAlexMG. Tous droits réservés.

```
dbt docs generate --static
REM Attendu : Building catalog, puis Catalog written to …\02_ae\target\catalog.json.
start 02_ae\target\static_index.html
REM Attendu : la documentation s'ouvre dans le navigateur.
```

Chercher `fct_ventes` dans la page : sa description, ses colonnes avec celle de `cle_membre`, ses tests, son code et son graphe de lignage, qui remonte jusqu'aux sources. `dbt docs generate` a écrit trois fichiers dans `02_ae\target` :

Le fichier	Ce qu'il porte
`manifest.json`	tout ce que dbt sait du projet : chaque modèle, son SQL compilé, ses dépendances, ses tests, ses descriptions
`catalog.json`	ce que l'entrepôt sait de chaque objet : ses colonnes et leurs types
`static_index.html`	la page, qui embarque les deux

`manifest.json` est le seul endroit où le lignage du projet existe sous une forme qu'un programme peut lire. Le dossier `target` reste hors de Git : dbt le refait à chaque commande.

Étape 6 — Tout d'un geste, `dbt build` · Fiabiliser et faire tourner

`dbt build` fait tout, dans l'ordre du graphe : il bâtit chaque modèle, passe le snapshot et lance chaque test. Les tests d'un modèle passent juste après lui, avant les modèles qui le lisent, et un test qui échoue fait sauter tout ce qui vient après : un test en faute sur `int_ventes` laisserait `fct_ventes` et ses tests de côté, marqués `SKIP`. Le snapshot passe à sa valeur de départ, le 30 juin 2025 : c'est l'état de votre dernier regard, et il n'ajoute rien.

6.1 Tout lancer, dans la première fenêtre.

```
dbt build
REM Attendu : 26 of 26 : neuf modèles, le snapshot et 16 tests, puis Done. PASS=26 WARN=0 ERROR=0.
```

Étape 7 — Versionner · Fiabiliser et faire tourner

7.1 Valider les sources, les tests, le snapshot et le fichier de l'étoile.

```
git -C 02_ae add models snapshots tests
git -C 02_ae commit -m "Cours 05 : les tests, l'histoire des prix, la documentation"
REM Attendu : 6 files changed.
git -C 02_ae status
REM Attendu : nothing to commit, working tree clean.
```


© 2026 iAlexMG. Tous droits réservés.

À retenir

- Un test est une requête qui ne doit rien rendre : générique et déclaré en YAML (`unique`, `not_null`, `accepted_values` toujours avec son `not_null`, `relationships`), ou singulier et écrit en SQL, comme le témoin ticket par ticket, en jointure externe complète.
- Un snapshot garde l'histoire qu'une table écrase, une version par changement, datée du jour où il est entré en vigueur ; il ne voit que ce qu'il regarde, et dans l'ordre.
- Le prix d'époque se lit sur la version ouverte le jour de la vente, fin exclue ; `dbt build` bâtit, historise et teste d'un geste, et la documentation sort des descriptions du YAML.

© 2026 iAlexMG. Tous droits réservés.